GD/CCPROG3 Machine Project Specifications
1st Term, AY 2025-2026

Ang Mitsa

I. Introduction

The project for this term is based on a classic computer games developed by Shinichi Nakamoto and Shigeki Fujiwara and released in 1983 -- Bomberman. Through the years, several versions of this game have been developed, but in general, it would have the following key elements:

- The hero

  The main character, controlled by the player, navigates a maze of boulders and walls trying to reach a certain objective (i.e. kill the villain, reach the goal, look for an item, etc.). Our hero can leave different kinds of bombs to clear his way or defeat his enemies.

- The villains

  These are creatures or other characters whose main objective is to hunt our hero and defeat him.

- The Environment

  The maze is where our hero and their enemies play-in. The environment consists of basically 2 kinds of obstacles: boulders that can be destroyed and boulders/walls that cannot be destroyed. Hidden underneath the destroyable blocks are power-ups that allow our characters to become stronger.

II. The Challenge

Your objective as developers of a start-up indie gaming studio is to develop the best version of this game. You will go through the motions of becoming familiar with previous games similar to this project, proposing your own version, doing a simple prototype of the project, until producing a game that could be released to the public.

The project will be split into three (3) main phases:

1. Phase 1: Initial Game Proposal

   In this phase, each team will submit a proposal containing the details of the different elements of the game. This includes the following:

   - abilities of the hero and villain(s);
   - sample drawing/image of ALL game elements;
   - details of the different power-ups available to the characters;
   - game features (i.e. load, save, high score, etc.); and
   - sample screen designs

2. Phase 2 (MCO1): Preliminary Design

   In this phase, each team will prepare the details of the approved game design in phase 1. The team will submit a class diagram as well as other important design issues that will be considered for implementation.

3. Phase 3 (MCO2): Final product delivery
   Each team will present their final output.

III. Minimum Requirements

The **initial design** should be complicated enough to merit a go signal for development. The following elements should be present:

- One main hero
- At least 2 villains with different behaviors
- At least 2 kinds of walls (e.g., breakable and unbreakable)
- At least 2 kinds of powerups
- At least 1 map layout or design
- Copyright free assets obtained from other sources (Note: you have to cite these in your documentation)

The **final product** is expected to have:

- A graphical user interface
- Main hero is controllable by keyboard
- Other elements in the game behave as designed
- At least 3 playable maps or levels with increasing complexity

Some references for Bomberman can be found here:

- https://youtu.be/CZ9Pu9Usk5o?si=-gIWj0sxblmXKXJZ
- https://youtu.be/7gLt1l5Q-Ls?si=KtUbl47U-UFirYGd
- https://www.youtube.com/watch?v=zzdQJsbMumw

IV. Milestones (see section V for Deliverables)

a. Game proposal – due 9:00 pm, October 3 (F)

Game design document that contains the following sections:

1. Game Title
   Think of a catchy title. It should be unique. Check the internet if the title already exists or is being used.
2. Game elements
   2.1. Hero
   2.2. Enemies
   2.3. Environment
   For this section and subsections, describe all the elements in detail, particularly their abilities and behavior
3. Level Design
   Prepare the map layout for at least one level.
4. Control & User Interface
   Describe the keyboard controls. How will the player interact with the game? Provide a UI sketch – a simple drawing of what the player sees on screen for the final product (e.g., timer on the top-left, score on the top right, etc.).

5. Art & Sound Direction
   Provide the images that you will be using for the final product. If these assets are obtained from the internet, ensure that it is free to use. Cite sources and original authors.

6. Win/Lose Conditions
   Discuss how the player wins (e.g., reaching the end, defeating the final boss, etc.)
   Discuss how the player loses (e.g., running out of time, losing all lives, etc.)

b. MCO1 – due 9:00 pm, October 24, 2025 (F)

1. UML Class Diagrams for: Hero and Environment elements (e.g., bomb, walls, etc.).
2. Implementation of the class: Hero, Bomb, breakable and unbreakable walls.
3. A text-based program to test the basic game. That is, the hero should be able to go around the map, drop bombs, and destroy breakable walls. No GUI display is expected. However, displays must be made in the Console.

c. MCO2 – due 12:00 nn, November 24, 2026 (M)

1. UML Class Diagrams (Object-Oriented)
2. GUI
3. Complete implementation of the game
4. Using the Model-View-Control (MVC) design pattern.

V. Deliverables

The deliverables for both MCOs include:

1. The design and implementation of the solution should:
   - Conform to the specifications described above
   - Exhibit proper object-based / object-oriented concepts, like encapsulation and information-hiding, etc.
   - **NOT** be derived or influenced from any use of Generative AI tools or applications
2. Signed declaration of original work (declaration of sources and citations may also be placed here)
   - See **Appendix A** for an example
3. Softcopy of the class diagram following UML notations (in pdf or png)
   - Kindly ensure that the diagram is easy to read and well structured
4. For CCPROG3 students, Javadoc-generated documentation for proponent-defined classes with pertinent information
5. Zip file containing the source code with proper internal documentation
   - The program must be written in Java (for CCPROG3 students) or C++ (for GDPROG3 students).
   - Include external libraries that were used (e.g., JavaFX)
6. Test script following the format indicated in **Appendix B**
   - In general, there should be at least 3 categories (as indicated in the description) of test cases per method (except for setters and getters).
   - There is no need to test user-defined methods which are ONLY for screen design (i.e., no computations/processing; just print/println).
7. For MCO1 only: A video demonstration of your program

- While groups have the freedom to conduct their demonstration, a demo script will be provided closer to the due date to help with showing the expected functionalities.
- The demonstration should also quickly explain key aspects of the program's design found in the group's class diagram
- Please keep the demo as concise as possible and refrain from adding unnecessary information

8. Groups should back-up their projects. A softcopy of the final unmodified files (for each phase) should be sent to their own emails, apart from regular submissions of progress in AnimoSpace and/or Git.

VI. Submission

All deliverables for the MCO are to be submitted via AnimoSpace. Submissions made in other venues will not be accepted. Please also make sure to take note of the deadlines specified on AnimoSpace. Late submissions will not be accepted.

VII. Grading

For grading of the MCO, please refer to the MCO rubrics indicated in the syllabus.

VIII. Collaboration and Academic Honesty

This project is meant to be developed as a pair for both phases (i.e., MCO1 and MCO2) barring any reports of freeloading or decision by the pair to split. In exceptional cases, a student may be allowed by their instructor to work on the project alone; however, permission should be sought as collaboration is a key component of the learning experience. Under no circumstance will a group be allowed to work on the MCO with more than 2 members.

A student cannot discuss or ask about design or implementation with other persons, with the exception of the teacher and their groupmate. Questions about the project specifications should be raised in the Discussion page in AnimoSpace. Copying other people's work and/or working in collaboration with other teams are not allowed and are punishable by a grade of 0.0 for the entire subject and a case may be filed with the Discipline Office. In short, do not risk it; the consequences are not worth the reward. Comply with the policies on collaboration and AI usage as discussed in the syllabus.

IX. Documentation and Coding Standards

Do not forget to include internal documentation (comments) in your code. At the very least, there should be an introductory comment and a comment before every class and every method. For those implementing using Java, this will be used later to generate the required External Documentation for your project via javadoc. You may use an IDE or the appropriate command-based instructions to create the documentation, but it must be PROPERLY constructed.

Please note that you're not expected to provide comments for each and every line of code. A well-documented program also implies that coding standards are adhered to in such a way that they aid in the documentation of the code. Comments should be considered for more complex logic.

X. Bonus Points

Bonus points will only be awarded for MCO2. The above description of the program is the basic requirement. Any additional feature will be left to the creativity of the group. Bonus points would be awarded depending on the additional implemented features. These additional features could include:

- Multiplayer implementation via the network
- Released game on itch.io and played by at least 30 people
- Developing more than 5 levels (game levels should be designed properly)

Depending on the scale of the new feature, additional points will be awarded to the group. However, make sure that all the minimum requirements are completely and correctly met first; if this is not the case then no additional points will be credited despite the additional features. To encourage the usage of version control, please note that a small portion of the bonus points for MCO2 will be the usage of version control. Please consider using version control as early as MCO1 to help with group collaboration.

XI. Resources and Citations

All sources should have proper citations. Citations should be written using the APA format. Examples of APA-formatted citations can be seen in the References section of the syllabus. You're encouraged to use the declaration of original work document as the document to place the citations.

Further, this is to emphasize that you DO NOT need to create your own sprites (background pictures, walls, characters, bombs, etc.) for the game. You can just use what's available on the Internet and just include these into your project, just make sure to cite your sources.

XII. Demo

Demo for MCO1 is via a video submission. All members are expected to be present in the video demonstration and should have relatively equal parts in terms of the discussion. Any student who is not present during the demo will receive a zero for the phase.

In MCO2, demo is live and will include an individual demo problem. Schedule for the demo will generally be during class time, but there may be other schedules opened by the faculty in case there is not enough time to accommodate everyone. Sequence (of who goes first in the class to do the demo) is determined by the faculty. Thus, do not be absent or late during announced demo days. A student or a group who is not present during the demo or who cannot answer questions regarding the design and implementation of the submitted project convincingly will incur a grade of 0 for that project phase.

During the demo, it is expected that the program can be compiled successfully in the command prompt and will run. If the program does not run, the grade for that phase is 0. However, a running program with complete features may not necessarily get full credit, as implementation (i.e., code) will still be checked.

XIII. Other Notes

You are also required to create and use methods and classes whenever possible. Make sure to use Object-Based (for MCO1) and Object-Oriented (for MCO2) Programming concepts properly. No brute force solution.

Statements and methods not taught in class can be used in the implementation. However, these are left for the student to learn on his or her own.

Appendix A. Template for Declaration of Original Work

Declaration of Original Work

We/I, [Your Name(s)] of section [section], declare that the code, resources, and documents that we submitted for the [1st/2nd] phase of the major course output (MCO) for GD/CCPROG3 are our own work and effort. We take full responsibility for the submission and understand the repercussions of committing academic dishonesty, as stated in the DLSU Student Handbook. We affirm that we have not used any unauthorized assistance or unfair means in completing this project.

[*In case your project uses resources, like images, that were not created by your group.*] We acknowledge the following external sources or references used in the development of this project:

1. Author. Year. Title. Publisher. Link.
2. Author. Year. Title. Publisher. Link.
3. Author. Year. Title. Publisher. Link.

By signing this declaration, we affirm the authenticity and originality of our work.

Signature and date	*Signature and date*
Student 1 Name ID number	Student 2 Name ID number

[*Note to students:*

1. *Do not submit documents where your signatures are easily accessible. Ideally, submit a flattened PDF to add a layer of security for your digital signatures*
2. *You may use the eSignature feature of Google Docs. See this template.*]

Appendix B. Example of Test Script Format

Class: MyClass						
Method	**#**	**Test Description**	**Sample Input Data**	**Expected Output**	**Actual Output**	**P/F**
isPositive	1	Determines that a positive whole number is positive	74	true	true	P
	2	Determines that a positive floating point number is positive	6.112	true	true	P
	3	Determines that a negative whole number is not positive	-871	false	false	P
	4	Determines that a negative floating point number is not positive	-0.0067	false	false	P
	5	Determines that 0 is not positive	0	false	false	P